# SCORE: Synthesising Complete, Owned, Responsive Environments for Simulation

J.P. Kardolus, Babon Innovations B.V., Utrecht, The Netherlands

Working draft, 27 October 2026

**Abstract**

Games, film, robot learning and embodied AI all depend on 3D environments that can be simulated, in which every object is a body of its own with collision, mass and joints, stands under the gravity of the ground beneath it, can be changed afterwards, and belongs to the person who made it. Current generative methods rarely deliver this: world models produce frames that no simulator can step, and one-shot 3D generators produce a single fused scene without separate bodies or clear rights to the result. We present SCORE, a framework that synthesises such environments by building them the way film studios build scenes in OpenUSD. Each scene sits within its parent scene, and each kind of work on it (layout, surfaces, physics, light, motion, effects, sound and people) is a layer of its own that is checked and changed independently of the others. The creator locks the design from the top down, first an environment concept that fixes its style and its physics and then a concept for each scene; open vision foundation models, procedural tools and coding agents then make every layer below, each checked by measured gates before the next is made. Every model in the framework allows commercial use of its output, so the creator remains the author, and every step is keyed on its inputs, so an edit reruns only the steps that depend on it. The result is a single OpenUSD environment whose physics a simulator reads as written; Newton, with MuJoCo's solver, steps it, and each character in it receives a sim body on Newton driven by its SOMA segment rotations. We describe the framework and its method, with a first environment as the evidence; its four demonstrations are marked as gaps until its batch has run. We close with the route toward simulated bodies that are true to the individual person and trained models that drive them. Code and paper are available at https://github.com/Babon-Innovations-b-v/score (MIT licence).

**Gap: Figure 1, one environment, every layer.** Environment 1 from the top down: its physics (the bodies it stands on and their gravity), a top scene on its site, a parent scene with its child scenes, and one scene opened into its layers side by side (base, settle, physics, light, motion, effects, sound, people). It waits on environment 1's batch.

## 1 Introduction

Simulated 3D environments have become raw material for games, film, robot learning and embodied AI [1]. What these uses have in common is not a particular look but a structure: separate objects with collision, mass and joints, ground with its own gravity, light and sound that come from a source in the scene, and a scene that can be opened, changed and simulated again. Building such an environment by hand is expensive, and this has made generative methods attractive; yet a generator that takes over the work must leave the result in its maker's hands, and many of those makers are sceptical, with 52 percent of game workers in GDC's 2026 survey reporting that they thought generative AI was hurting their industry [2].

The generative methods available today fall short of that structure in different ways. World models generate an environment as a sequence of frames [3], [4], which no simulator can step. 3D environment generators [5], [6] produce geometry, but mostly as one fused asset from which separate objects have to be recovered afterwards [7], [8]. Both work in a single shot, so a fault in the result cannot be traced back to the step that caused it. Procedural systems [9] and coding agents that write scenes as programs [10], [11] keep the environment explicit, but they rebuild scenes that already exist or draw from a fixed library, and their objects often float or come apart [12]. Open vision foundation models [13] can now read shapes [14], masks [15] and depth [16] from pictures, and a coding agent can connect them as tools, but none of them keeps one style across objects that different models made [17], [18], and none of them provides physics.

The pieces needed for the physics do exist. OpenUSD [19], the scene description Pixar developed for its films, keeps each department's work in a layer of its own and assembles scenes from smaller ones by reference, and its physics schema states bodies, colliders, mass and joints within the same stage. MuJoCo [20] is a widely used solver for articulated bodies in contact, and Newton [21], built on Warp [22], runs MuJoCo's solver on the GPU [23] and reads OpenUSD directly. What is missing is a way to get from what a creator wants to a complete environment in that form.

We observe that an environment need not be generated in one shot to be generated automatically. Studios already build scenes as a tree of smaller scenes, each a stack of layers that one department makes and another can change without touching the rest, and the same structure lets a machine build an environment one checked step at a time. If every layer is made from a design the creator has locked, by a step whose output is measured before anything is built on it, then a fault has an address, an edit has a bounded cost, and the physics can be written into the record itself rather than recovered from it.

This paper contributes not a new generative model but a framework that turns a creator's locked design into one OpenUSD environment which a simulator steps as written. What is new is the combination of three elements: a design locked from the top down, from an environment concept with its style and physics to a concept for every scene, from which every layer is read; layers made by keyed steps, each held to measured gates, so that an edit reruns only what it changes; and a record whose bodies, joints and gravity are written in OpenUSD's physics schema, made only with models that allow commercial use of their output. The environments this yields are what the name SCORE describes. They are complete, in that every object, surface, light and sound a scene shows exists wherever the viewer looks; owned, because every model in the framework allows commercial use of its output, so the creator holds the full rights; and responsive, because every object has its collider, mass and material and what moves has its joints. They are also editable, since each scene is a stack of layers and each layer is made by a keyed step. An environment can be as small as a house on a plot of ground or as large as a system of worlds that move one another.

Section 2 describes the framework, Section 3 its method, the stages that build a scene from the top down and layer by layer, and Section 4 the evidence, four demonstrations on a first environment that remain gaps until its batch has run. Section 5 discusses what the design implies and where it is limited, Section 6 positions it against related work, and Section 7 sets out where the framework goes next, toward simulated people who are true to the individual person.

## 2 The framework

An environment in SCORE is one OpenUSD record. Each scene is one stage, which sits within its parent scene and holds its child scenes, each referenced at the room its parent's plan gives it, so that opening a top scene opens everything below it. Within a scene the layers stack in a fixed order: a slot for the creator's edit layer on top, then people, sound, light, motion, effects, crops, machines, physics, settle and ground, and at the bottom the base laid from the locked plan. Each object corresponds to one real-world object and carries its kind, its transform, its collider and a library surface for each of its parts.

The physics of an environment is decided in its environment concept (EC). It may be a plot of ground with a gravity and a path of the sun, one world, or a system of bodies that move one another; a real body takes its values from a cited table, and an invented one is given by its radius, mass, orbit and spin. The physics layer writes each scene's bodies, masses, joints and the gravity of the body it stands on in OpenUSD's physics schema, so that any reader of that schema obtains the same scene.

The creator works in the creator cockpit, a single signed-in web page in which he locks the design, buys paid work and reviews what the framework reports. Beneath that page, an environment is a table of stages read off its designs; for environment 1 the table has 261 rows. A node is one stage applied to one scene (for example `assembly@hangar`) or to the environment as a whole, and a single runner makes, judges and accepts every node. Once a design is approved, everything runs without a further yes from the creator.

Three mechanisms make this safe to leave running. The first is the key. A node is made only when its key has no finish record, and the key hashes the stage and scene, its code version, contracts, parameters and seed, the pinned model images, every file the node reads and the accepted output of every node it reads. The code version counts only code that can shape the output; code that decides where and how a job runs changes what a run costs but never what it makes, and is therefore kept out of the key. An edit or a fixed fault moves only its own key and those downstream of it, so only those steps run again, and outputs are kept by content hash and never overwritten.

The second is the gate. Each gate answers pass, fail or unknown, together with its measures and a reason; missing evidence counts as unknown, and unknown blocks a node just as fail does. The runner accepts an output by itself when every gate has passed and the output is no worse than the one accepted before it, and later nodes read only accepted outputs. The creator keeps four actions for himself: he may reject an accepted output, which makes every node that read it stale; accept a worse one with a reason; waive one gate for one key; and override the runner for one paid run. A store that refuses these records when they come from an agent's process, together with a hook, keeps agents from taking these actions.

The third is the batch. Paid work is bought as a batch with a single estimate, and the creator's approval sets one budget for the batch across all its tries, at which the money guards stop it. Within a batch every paid job runs once, and an output that fails becomes a batch gap that the completion gate lists. Every paid stage is priced from a unit cost measured on the ledgers; a stage that has never run has no measured price, so its estimate is shown separately as a guess.

# 3 Method: building an environment

The method has two parts. The first turns what the creator wants into a locked design: a concept for the environment and for each of its scenes, an inventory of every object, and a plan in metres. The second reads the layers each scene needs off that design and makes each in turn, behind its gates, until the scene passes its completion gate.

## 3.1 From concept to plan

Step 1: the environment concept. The creator first locks the EC in a short interview with Gemini 3.1 Pro [24], in which he fixes its style and its physics, adds his reference pictures and names the scenes it holds. An environment plan then places each top scene's site on that ground and works out the sky each site sees.

Step 2: a concept for each scene. A concept is then locked for every scene, in the order of parent scene before child scene. Each draw is one Nano Banana Pro [25] call, made on a concept review page where the creator can mark areas and ask for a new draw. A child scene is drawn only once its parent has been locked and planned, and it is drawn on a crop of the parent's concept, so that it fits the parent it stands in.

Step 3: inventory and plan. Gemini 3.1 Pro lists every object of the locked concept as a row of the scene inventory, giving its kind, count, anchor and size in metres, and SAM 3 [15] draws a box around each row; nothing that is not a row is made or placed. Nano Banana Pro then draws the scene from straight above, and Gemini 3.1 Pro writes the plan in metres from that drawing, the concept and the inventory. The plan gives the outline, walls, openings, levels, stairs, roofs and terrain, each item with its facing, people as the spots on which they stand, and the rooms of the child scenes. A set of free checks follows: every row must be placed as often as its count says, lie inside the outline and not overlap another floor item, every door and standing spot must be reachable on foot, and the child scenes must tile their parent. What fails is returned to the model as notes at most twice, and only what is left after that becomes a gap. The creator's own edits on the plan review page always take precedence and redo what follows from them.

The route is the same loop at every scale, so that shapes are fixed before detail. A parent scene receives its concept, its plan and a rough model; its plan is then split into the shapes of its child scenes, and once those shapes are fixed every child scene is made in parallel, each repeating the loop. The two stages that carry this, `rough-model` and `shell-split`, are built but have not yet run.

## 3.2 Layers from row kinds

A scene's layers are read off its design alone. Each inventory row's kind, found from the noun the reading gave it by a single table of rules, says what its objects do: emit, move, lie loose, glow, grow, act or sound. A scene receives an effects, motion, physics, crops or people layer only when it has rows that do so, and always receives light and sound. Each layer is made from its rows by the rule for their kind, so a new environment needs no hand-written list of what to build. The scene record holds what no made object covers (shells, stairs, ground and water, backdrop, lamps and neighbouring buildings), and it is gated on the shell lying within 5 cm of the plan and on every doorway opening onto a floor.

Models are made by the prop route first. Nano Banana Pro draws a close-up of each object from its crop of the concept, redrawn at most three times when the shape check rejects it; Pixal3D [14]

builds the model from that close-up; and a likeness gate compares the model's outline from eight sides with the close-up. Only an object whose checks fail is built instead by a code builder that a coding agent writes, and a thin straight object (a pipe, post or beam) whose every close-up has failed is swept along its length in code.

Surfaces come from the creator's library. Everything a scene shows is drawn from score_lib or recorded as a written exception, and a model contributes its shape only. Each of its parts takes one surface from a library of rule-based surfaces, written as ProcFunc functions [26] on Infinigen's shaders [9] and coloured from the scene's palette. The finish route labels each face from free renders of the model, matches each finish to a library surface, bakes it, and checks the bake against the close-up, requiring each finish to lie within $\Delta$E 32 of its colour there. A free library check then traces every material, model, light, sound and character of an accepted assembly back to the library or to an exception.

### 3.3 Settling and physics

Loose objects are dropped onto the made ground, and the settle writes where each comes to rest as a layer of its own; an object that would turn by more than 15 degrees or drift by more than 30 cm is recorded as a resting fault and keeps its laid pose. The physics trial then steps the composed scene on Newton, using MuJoCo's solver on Newton's contacts under the gravity of the body the scene stands on, and judges it by four gates: `reacted`, `at rest`, `energy`, which requires that undriven energy does not rise, and `hinge`, which requires a hinged leaf to swing to its limit.

The trial moved from Blender's Bullet [27] to Newton on 27 October 2026, and the move was checked on a toy scene at Moon gravity with two crate drops, a driven door with a wheel fixed to it and a pushed stool. Both engines gave the same answer on every gate, and Newton landed the crates on their targets where Bullet left them on its 8 mm contact margins. Newton's deterministic mode was bitwise equal over six runs on that toy, but it cannot hold scenes with many loose bodies, because its buffers then pass the 32-bit limit; the trial and the settle therefore run without it, and two runs of a 25-body toy agree within 5e-7 m on every body except one that falls off the edge of the floor. The settle is moving to Newton in the same way, so that one engine steps both what comes to rest and what is tried, and one effect is to be stepped on Newton as a first case for the effects layer.

### 3.4 Light, motion, effects and sound

The sky is computed from the EC's physics rather than drawn, with each body placed at its true direction, size and phase as seen from the scene's site. The lamps are the inventory's glowing rows: one render per scene gives each lamp its own light group, and a free sweep mixes their strengths until the scene lies within 1.5 times its concept's brightness. Every object of a moving row moves by the rule for its kind, at a speed cited from real machines, and is gated on nothing passing within 3 cm of anything else. Emitters are measured against real footage, and each sound source receives either a library file or one MOSS-SoundEffect [28] take, gated on a CLAP [29] score of at least 0.30.

### 3.5 People and sim bodies

People are made like any other model, from a close-up per person row that Pixal3D builds and the finish route paints. SOMA's body [30], read from the close-up by SAM 3D Body [31], is fitted beneath the model as its rig, and Kimodo's clips [32] move it; a shell gate passes a look when 99 percent of its soft points lie within 5 cm of the body. Each person stands on a person spot of its scene's plan.

Each character also receives a sim body, its physical twin on Newton, so that the people in an environment are simulated and not only animated. The minimal body is built directly from SOMA's 23 segments, with a ball joint at each SOMA joint frame, and is driven by the character's SOMA segment rotations without scaling or per-person setup. Its mass is distributed by segment volume, and it stands on a SCORE floor, so its ground forces come from contact; muscles enter as a minimal first step. Built in this way, the body follows SOMA's rotations to within 6.9e-9 m, and on Newton it matches plain MuJoCo to within 3.4e-8 after 200 steps.

**Gap: Newton for the settle, one effect and the sim body.** The settle and the physics trial on one engine, with the resting and physics gates per scene against the Bullet settle; the one effect stepped on Newton; and a sim body walking on a SCORE floor, its joint angles and ground forces against OpenCap's LabValidation motion capture and force plates [33]. It waits on these three stages being built and run on a cloud card.

### 3.6 Completion

A scene is complete when its completion gate passes on its final assembly, which requires every row to be made or dropped with a reason, nothing to be left unmade, no resting fault, and every layer and every shown scene to be present. A fault the creator decides to ship with becomes an accepted gap, recorded with who decided, when and why; the gate lists every gap by name and is never loosened. A hook stops an agent from claiming that a scene is done while its gate fails, because agents judging their own scenes agree with a measured geometry score only 45.8 percent of the time [34]. Once a scene is complete, its films are made from scripts the creator has locked.

## 4 Evidence: environment 1

Environment 1 is the evidence for the framework. Its first run was not accepted by the creator, and its store was lost on 19 October 2026, so this draft shows none of its outputs; each demonstration below will be made on its new batch, Figure 1 being the first.

**Gap: Demonstration 2, physics that holds.** Environment 1's scenes stepped on Newton under their own body's gravity: loose objects come to rest, doors swing on their hinges, with the physics trial's gates per scene and Bullet run on the same scenes as the baseline. It waits on environment 1's batch and on Newton taking over the settle.

**Gap: Demonstration 3, one environment, many uses.** The same OpenUSD environment opened unchanged in a game engine, in a robot simulator and in a film render, with what each reads of its layers and what it drops. It waits on environment 1's batch and on the engine adapter.

**Gap: Demonstration 4, an edit that stays cheap.** The creator changes one thing in a locked scene, and only the nodes whose keys it moves run again; the nodes rerun, time and euros against a full rebuild. It waits on environment 1's batch and on the edit layer being carried from one assembly to the next.

**Gap: Table 1.** Per scene of environment 1: inventory rows, models generated and code-built, layers, gaps before and after the batch, time, euros and the creator's verdict. It waits on environment 1's batch and the creator's review.

The division of labour behind this work is as follows. The creator's locks, picks, accepts and waivers are his own clicks, and everything between them runs automatically. Coding agents wrote most of the framework's code under the author's direction: 2,413 of its 2,436 commits carry a coding agent as co-author. Hand fixes to a scene are recorded as the creator's edits.

# 5 Discussion

The design rests on one observation, that an environment generated step by step, with every step held to a measured gate and keyed on its inputs, can carry its physics and its authorship in the record itself. The consequences of this are concrete, although until environment 1's batch has run they are properties of the design and not yet measured results:

- **Faults have an address.** Because each layer is made by its own node and judged by its own gates, a fault is found at the step that caused it and not in a finished scene, and fixing it moves only the keys downstream of that step. Demonstration 4 is to measure what this saves.
- **One record for many readers.** The physics is written in OpenUSD's physics schema and not in one engine's format, so any reader of that schema obtains the same bodies, joints and gravity. Demonstration 3 is to test this in a game engine, a robot simulator and a film renderer.
- **The creator stays the author.** Every model allows commercial use of its output, every finish traces to the creator's library, and the decisions that shape the result (locks, accepts, rejects and waivers) are his own, recorded with their reasons.
- **Agents are not their own judges.** A coding agent writes code builders and connects tools, but it never decides that its own work is done; that decision belongs to measured gates and, above them, to the creator.

## 5.1 Limitations

**One creator and no accepted run.** A single creator judges every environment, and he has not yet accepted a run; the method has therefore not yet been shown to produce an environment its maker would ship.

**Closed models.** Nano Banana Pro and Gemini 3.1 Pro draw the concepts and close-ups and read and plan every scene. They are closed and may change or be withdrawn, and replacing them with open models remains to be done.

**Unexercised stages.** The design step, the layers from row kinds, the finish route, the shapes-first loop and the scene tree have not yet carried a scene through a batch. The edit layer is still a slot, the settle, the effects and the sim body are not yet on Newton, and no game engine or robot simulator has yet opened the result.

# 6 Related work

SCORE sits between three lines of work. World models [3], [4] and 3D environment generators [5], [6] generate an environment in one shot, as frames or as a fused asset, and recent work recovers separate objects from that asset afterwards [7], [8]; SCORE instead makes every object separately from the start, and writes its body into the record. Procedural systems [9], [35] keep scenes explicit through hand-written generators, and SCORE uses their shaders, through ProcFunc [26], as the base of its surface library, while reading what to build from a creator's concept rather than from a fixed grammar. Coding agents that write scenes as programs [10], [11], [36], [37], [38], [39] are closest in spirit, but the scenes they write often float or come apart [12], they judge their own geometry unreliably [34], and on scenes rebuilt from video the strongest model scores 0.91 on static scenes but only 0.67 on dynamic ones [40]. SCORE therefore gives such agents a narrow role, writing builders for the objects other routes fail on, and leaves judgement to measured gates.

## 7 Future work: people true to the person

The minimal sim body is a starting point, and this section states goals rather than results.

The sim body is to become fully musculoskeletal and true to the individual person, older people and those with a curved spine included, rather than a template body resized to fit. Its joint centres are to come from bone landmarks on its SOMA mesh, followed by its mass, muscles, joint ranges, spine and bone shape, with each input that SOMA lacks marked as assumed and given a range. A full musculoskeletal body with 416 muscles already runs on Newton, but it still drifts 1.5 cm from plain MuJoCo after 300 steps, so Newton is to be validated for bodies before more is built on it. The body's joint angles, and then its joint moments and ground forces, are to be held to OpenCap's LabValidation data.

Motion is to come from the SOMA ecosystem: from video through GEM-X [41], from sentences through Kimodo, and from sequences joined across datasets of both. The terrain a motion implies then becomes a SCORE scene on which a sim body follows the motion, so that the scene keeps the motion without the video and without anything that identifies a person.

Beyond following recorded motion, we intend to train a brain, a model that drives a sim body's muscles so that it moves on its own, including over structures it has never seen. Trained on sim-body data and placed in SCORE's invented worlds, under their own gravity and ground, a brain is to adapt to situations no recording holds. Specialised models, for example for clinical groups, would be trained outside SCORE on data that never enters it. Together these form a data loop: sim bodies moving in synthesised scenes produce data, that data trains better and more specialised brains, and a brain driving a sim body in new scenes produces more data.

## 8 Conclusion

Simulable environments have been expensive to make because each object, surface, light, sound and body had to be built and joined by hand, and the generative methods that promise to remove that cost produce frames or fused geometry that a simulator cannot step and a creator cannot fully own. SCORE takes a different route. It builds an environment the way studios build a scene, scene within scene and layer by layer, from a design the creator has locked, with every layer made by a keyed step and held to measured gates, and it writes the result as one OpenUSD record whose physics a simulator steps as written. The next steps are environment 1's batch and its four demonstrations, an engine adapter, the edit layer carried across assemblies, further environments, and open models in place of the closed ones. If an accepted environment confirms that the approach holds, we hope it brings owned, simulable worlds within reach of individual creators, researchers and small teams who could not afford to build them by hand, and gives simulated people a place to move in.

## Author Contributions

J.P. Kardolus is the sole author. He conceived the framework, directed its development, made every design lock, pick and accept, and wrote the paper. Coding agents wrote most of the code and drafted the text under his direction, as disclosed in Section 4.

## Competing Interests

The author is affiliated with Babon Innovations B.V., in whose GitHub organisation SCORE is published under the MIT licence. The author declares no competing interests.

## Acknowledgments

We thank the authors and maintainers of the open tools and data this work builds on: OpenUSD [19]; Newton [21], Warp [22], MuJoCo [20] and MuJoCo Warp [23]; Blender [42] and Bullet [27]; Infinigen [9] and ProcFunc [26]; SAM 3 [15], SAM 3D Body [31] and Pixal3D [14]; SOMA [30], Kimodo [32] and GEM-X [41]; MOSS-SoundEffect [28] and CLAP [29]; and the OpenCap team for the LabValidation data [33].

## Data Availability

The framework's code, the source of this paper, its PDF and its arXiv tree are public at https://github.com/Babon-Innovations-b-v/score under the MIT licence. The run records from which the paper's numbers are read are in the repository under `paper/evidence/`, and every number in the source carries a comment naming the file, commit or issue comment it is read from. Environment 1's outputs are not yet made and are marked as gaps. OpenCap's LabValidation data (Apache-2.0) is fetched from its own source when needed and is not redistributed here.

## References

[1] Y. Yang *et al.*, "Holodeck: Language guided generation of 3D embodied AI environments," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[2] D. Argüello, "36 percent of game workers use generative AI, but half think it's bad for the industry." Game Developer, Feb. 2026.

[3] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, *et al.*, "Genie: Generative interactive environments," *arXiv preprint arXiv:2402.15391*, 2024.

[4] J. Parker-Holder and S. Fruchter, "Genie 3: A new frontier for world models." Google DeepMind blog, Aug. 2025. Available: https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/

[5] D. Wang, H. Jung, T. Monnier, K. Sohn, *et al.*, "WorldGen: From text to traversable and interactive 3D worlds," *arXiv preprint arXiv:2511.16825*, 2025.

[6] Tencent Hunyuan, "HY-World 2.0." GitHub repository, 2026. Available: https://github.com/Tencent-Hunyuan/HY-World-2.0

[7] J. Hu, J. Guo, J. Cen, C. Yang, S. Li, and W. Shen, "WorldAct: Activating monolithic 3D worlds into interactive-ready object-centric scenes," *arXiv preprint arXiv:2605.15843*, 2026.

[8] M. Niu, J. He, R. Yu, L. Fu, *et al.*, "WorldSculpt: Generating compositional worlds from grounded videos," *arXiv preprint arXiv:2609.05416*, 2026.

[9] A. Raistrick *et al.*, "Infinite photorealistic worlds using procedural generation," *arXiv preprint arXiv:2306.09310*, 2023.

[10] Z. Hu, A. Iscen, A. Jain, T. Kipf, *et al.*, "SceneCraft: An LLM agent for synthesizing 3D scene as Blender code," in *International conference on machine learning (ICML)*, 2024.

[11] H. Wang *et al.*, "Code as worlds: Agentic discovery of executable world representations for physical reasoning," *arXiv preprint arXiv:2608.27549*, 2026.

[12] Y. Gao *et al.*, "3DCodeBench: Benchmarking agentic procedural 3D modeling via code," *arXiv preprint arXiv:2606.01057*, 2026.

[13] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[14] D.-Y. Li *et al.*, "Pixal3D: Pixel-aligned 3D generation from images," *arXiv preprint arXiv:2605.10922*, 2026.

[15] N. Carion *et al.*, "SAM 3: Segment Anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.

[16] R. Wang *et al.*, "MoGe-2: Accurate monocular geometry with metric scale and sharp details," *arXiv preprint arXiv:2507.02546*, 2025.

[17] J. Wu, Z. Lou, J. Liu, D. Du, C. Guo, and S. Guo, "From visual synthesis to interactive worlds: Toward production-ready 3D asset generation," *arXiv preprint arXiv:2604.23629*, 2026.

[18] Z. Yang *et al.*, "FlowScene: Style-consistent indoor scene generation with multimodal graph rectified flow," *arXiv preprint arXiv:2603.19598*, 2026.

[19] Pixar Animation Studios and Alliance for OpenUSD, "OpenUSD: Universal scene description." 2026. Available: https://openusd.org/

[20] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ international conference on intelligent robots and systems (IROS)*, 2012, pp. 5026–5033.

[21] The Newton Developers, "Newton: A GPU-accelerated physics simulation engine." 2026. Available: https://github.com/newton-physics/newton

[22] M. Macklin, "Warp: A high-performance Python framework for GPU simulation and graphics." NVIDIA, 2022. Available: https://github.com/NVIDIA/warp

[23] Google DeepMind and NVIDIA, "MuJoCo Warp: A GPU-optimized version of the MuJoCo physics simulator." 2026. Available: https://github.com/google-deepmind/mujoco_warp

[24] Google, "Gemini 3.1 Pro (gemini-3.1-pro-preview) through the gemini API." 2026. Available: https://ai.google.dev/

[25] Google, "Gemini 3 Pro Image ('Nano Banana Pro') through the Gemini API." 2026. Available: https://ai.google.dev/

[26] A. Raistrick *et al.*, "ProcFunc: Function-oriented abstractions for procedural 3D generation in Python," *arXiv preprint arXiv:2604.26943*, 2026.

[27] E. Coumans and Y. Bai, "Bullet physics SDK and PyBullet." 2021. Available: https://github.com/bulletphysics/bullet3

[28] OpenMOSS-Team, "MOSS-SoundEffect v2.0." Hugging Face, 2026. Available: https://huggingface.co/OpenMOSS-Team/MOSS-SoundEffect-v2.0

[29] Y. Wu, K. Chen, T. Zhang, *et al.*, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," *arXiv preprint arXiv:2211.06687*, 2022.

[30] J. Saito, J. Li, M. de Ruyter, M. Guerrero, *et al.*, "SOMA: Unifying parametric human body models," *arXiv preprint arXiv:2603.16858*, 2026.

[31] X. Yang, D. Kukreja, D. Pinkus, others, and K. Kitani, "SAM 3D Body: Robust full-body human mesh recovery," *arXiv preprint arXiv:2602.15989*, 2026.

[32] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, *et al.*, "Kimodo: Scaling controllable human motion generation," *arXiv preprint arXiv:2603.15546*, 2026.

[33] S. D. Uhlrich *et al.*, "OpenCap: Human movement dynamics from smartphone videos," *PLOS Computational Biology*, vol. 19, no. 10, p. e1011462, 2023.

[34] X. Li *et al.*, "LEGO-Anything: Coding agents for 3D scene reconstruction," *arXiv preprint arXiv:2609.36380*, 2026.

[35] A. Raistrick *et al.*, "Infinigen Indoors: Photorealistic indoor scenes using procedural generation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[36] Z. Li, Y. Liao, and B. Zhu, "Recursive code world models: Building complex worlds through recursive scene programs," *arXiv preprint arXiv:2609.11499*, 2026.

[37] C. Guo, J. Li, Y. Li, and Z. Huang, "WorldClaw: Agentic 3D open-world generation at scale," *arXiv preprint arXiv:2608.05248*, 2026.

[38] L. Yin, W. Cheng, Z. Qin, T. Huang, *et al.*, "AutoUE: Automated generation of 3D games in Unreal Engine via multi-agent systems," *arXiv preprint arXiv:2603.07106*, 2026.

[39] G. He, R. Luo, W.-C. Ma, and H. Averbuch-Elor, "Thinking in Blender: Staged executable inverse graphics with vision-language models," *arXiv preprint arXiv:2606.02580*, 2026.

[40] R. Shen *et al.*, "4DCodeBench: Benchmarking agents on inverse graphics of dynamic scenes," *arXiv preprint arXiv:2610.03715*, 2026.

[41] NVIDIA, "GEM-X: Monocular whole-body 3D human pose estimation using the SOMA body model." 2026. Available: https://github.com/NVlabs/GEM-X

[42] Blender Foundation, "Blender." 2026. Available: https://www.blender.org/